

华东师范大学数据学院上机实践报告

课程名称：操作系统

年级：2022 级

上机实践成绩：

指导教师：翁楚良

姓名：

上机实践名称：实验四 Locks

学号：

上机实践日期：2024.6.13

一、实验目的

- 学习自旋锁和睡眠锁
- 优化 block cache 争用
- 改进 xv6 操作系统的缓冲区缓存（buffer cache），以减少在多进程频繁使用文件系统时对 bcache.lock 的争用。

二、实验内容

整体的实验解决思路是将资源进行拆分，需要使用哈希表，哈希表中设置素数(实验提示使用 13 这个素数)个桶 bucket，每一个桶内设置放置一些 buffer，并且每一个桶都有一个独立的锁。那么当使用 bget 获取 buffer 时，只需要将磁盘块的编号进行 hash，返回一个整数作为哈希表内桶的编号，并在桶中找到指定的 buffer。

原始版本的 buffer cache 由一个大锁 bcache.lock 保护，限制了并行运行的效率，且使用的是双链表的管理方式。

- 修改 kernel/bio.c 文件：放弃双链表的管理方式，利用 hash bucket 思想修改缓冲区缓存，以减少对 bcache.lock 的争用。使用哈希表在缓存中查找块号 block number，哈希表的每个哈希桶都有一个锁。

- 删除所有缓冲区的列表（如 bcache.head 等），改为使用缓冲区上次使用的时间戳（即使用 kernel/trap.c 中的 ticks）。通过这种改变，brelse 不需要获取 bcache 锁，bget 可以基于 ticks 选择最近最少使用的块。

- 运行 bacheltest 时，bcache 中所有锁的 acquire 循环迭代次数接近于零。理想情况下，块缓存中涉及的所有锁的计数之和应该为 0，本实验允许总和小于 500。保持每个块最多只有一个缓存副本的不变性。最终运行 bacheltest 和 usertests 时保持所有测试通过。

三、使用环境

VMware Workstation Pro

四、实验过程及结果

- 自旋锁和睡眠锁都作用于同步机制，保证数据互斥访问，保持数据一致性。

自旋锁在被占用后，线程会一直轮询访问锁，直到被释放，适用于短时间内可以获取到锁的场景；睡眠锁在被占用后，线程会阻塞，直到该锁被释放，适用于长时间获取不到锁的场景，因为线程阻塞后，长时间内不会占用 CPU 时间片资源。

- 函数结构

在 bio.c 中 binit() 具有对当前缓冲区初始化的作用，对缓存区的每一个 buffer 初始化一个睡眠锁，对整个缓存区初始化一个自旋锁，从前往后形成一个双向链表，初始时缓存中每一个块都为空。

上层日志调用 bread() 函数对 buffer 进行读操作，通过 bget() 根据 blockno. 来请求当前需要的磁盘块是否在缓存中，如果在则直接返回 buffer 块，如果不在则用 LRU 算法返回最近最少使用的块重新进行读写。

bwrite() 将缓存中的数据更新并重新写入磁盘中。

brelse() 释放不再使用的缓存区。

在 buf.h 中新增 lastuse 表示最近使用时间。

```
C buf.h
1  struct buf {
2      int valid;    // has data been read from disk?
3      int disk;    // does disk "own" buf?
4      uint dev;
5      uint blockno;
6      struct sleeplock lock;
7      uint refcnt;
8      struct buf *prev; // LRU cache list
9      struct buf *next;
10     uchar data[BSIZE];
11     uint lastuse; //
12 };
```

原始的 buffer cache 使用的是双向链表，我们需要把它改成 hash bucket，使用哈希表在缓存中查找 blockno。一开始定义哈希桶的数量和桶中的 buffer 数量，根据提示定义 BUCKETSIZE 为 13，BUFFERSIZE 为 5，表示 13 个桶分别有 5 个 buffer。

```

17 #include "types.h"
18 #include "param.h"
19 #include "spinlock.h"
20 #include "sleeplock.h"
21 #include "riscv.h"
22 #include "defs.h"
23 #include "fs.h"
24 #include "buf.h"
25
26 #define BUCKETSIZ 13 // number of hashing buckets
27 #define BUFFERSIZ 5 // number of available buckets per bucket
28
29 extern uint ticks; //引入时间戳

```

修改结构体为 bcache bucket。将原有的 head 删除，改成 13 个 bucket 结构体数组。

```

30 // struct {
31 //     struct spinlock lock;
32 //     struct buf buf[BUFFERSIZ]; //缓冲区数组
33 //
34 //     // Linked list of all buffers, through prev/next.
35 //     // Sorted by how recently the buffer was used.
36 //     // head.next is most recent, head.prev is least.
37 //     struct buf head; //双向链表的头节点
38 // } bcache;
39
40 //修改结构体bcache为bcachebucket
41 struct {
42     struct spinlock lock;
43     struct buf buf[BUFFERSIZ];
44 } bcachebucket[BUCKETSIZ]; //13个桶5个buffer

```

```

46 /*
47 //每个桶一个自旋锁，每个缓冲区一个睡眠锁
48 void
49 binit(void) //对缓冲区的初始化操作，创建一个循环链表管理缓冲区
50 {
51     struct buf *b; //buffer指针
52
53     initlock(&bcache.lock, "bcache"); //给整个缓冲区初始化一个自旋锁
54
55     // Create linked list of buffers
56     bcache.head.prev = &bcache.head;
57     bcache.head.next = &bcache.head; //初始化双向链表
58     for(b = bcache.buf; b < bcache.buf+NBUF; b++){
59         b->next = bcache.head.next;
60         b->prev = &bcache.head;
61         initsleeplock(&b->lock, "buffer"); //对每个buffer块加睡眠锁
62         bcache.head.next->prev = b;
63         bcache.head.next = b; //头插法
64     }
65 }
66 */

```

修改 binit(), 初始化 bcache 中的 bucket, 初始化每个 bucket 的自旋锁, 每个 bucket 的 buffer 的睡眠锁。

```

67 void
68 binit(void)
69 {
70     for(int i=0; i<BUCKETSIZE; i++){
71         initlock(&bcachebucket[i].lock, "bcachebucket"); //对每个桶初始化自旋锁
72         for(int j=0; j<BUFFERSIZE; j++){
73             initsleeplock(&bcachebucket[i].buf[j].lock, "buffer"); //对桶的buffer初始化睡眠锁
74         }
75     }
76 }

```

新增 hash 函数，采用模运算方式，将 blockno 转换为桶的编号。

```

116 int hash(uint blockno)
117 {
118     return blockno%BUCKETSIZE;
119 }

```

在 bget()函数通过 blockno 来请求当前所需要的磁盘块是否在缓存中，如果在就直接返回 buffer，如果不在就用 LRU 算法返回最近最少使用的空闲块。

首先根据 blockno 哈希计算 bucket 然后获取锁，然后遍历这个桶中的所有缓冲区，如果找到了匹配的缓冲区就增加此缓冲区的引用计数 refcnt，更新最近使用时间 lastuse，并直接返回这个缓冲区。

```

122 static struct buf*
123 bget(uint dev, uint blockno)
124 {
125     struct buf *b;
126     //申请桶的锁
127     int bucket = hash(blockno);
128     acquire(&bcachebucket[bucket].lock);
129
130     for(int i=0; i<BUFFERSIZE; i++){
131         b = &bcachebucket[bucket].buf[i];
132         if(b->dev ==dev && b->blockno == blockno){
133             b->refcnt++;
134             b->lastuse = ticks;
135             release(&bcachebucket[bucket].lock);
136             acquiresleep(&b->lock);
137             return b;
138         }
139     }

```

如果没有找到匹配的缓冲区，就要遍历查找该桶最近最少使用的空闲 buffer，即引用计数为 0 且 lastuse 最小，如果没有空闲块就报 panic，否则将引用计数记为 1，lastuse 记为时间戳 ticks，更新设备号和 blockno 后释放 bucket 的锁，获取 buffer 的锁后返回空闲 buffer 即可。

```

141 //遍历查找该桶最近最少使用的空闲buffer(ticks较小)
142 uint flag = 0xffffffff; //最大的unsigned int
143 int least_idx = -1;
144 for(int i=0; i<BUFFERSIZE; i++){
145     b = &bcachebucket[bucket].buf[i];
146     if(b->refcnt == 0 && b->lastuse < flag){
147         flag = b->lastuse;
148         least_idx = i;
149     }
150 }
151
152 if(least_idx == -1){
153     panic("bget: no unused buffer for recycle");
154 }
155
156 b = &bcachebucket[bucket].buf[least_idx];
157 b->dev = dev;
158 b->blockno = blockno;
159 b->lastuse = ticks;
160 b->valid = 0;
161 b->refcnt = 1;
162 release(&bcachebucket[bucket].lock);
163 acquiresleep(&b->lock);
164
165 return b;
166 }

```

在 log.c 中会调用 bread 和 bwrite，传入 dev 和 blockno，进行 bget 操作。如果 b 是空闲块，就要从磁盘中读取对应 blockno 的内容，然后写入当前的空闲块。然后因为已经读取数据，把 valid 修改为 1。

```

168 // Return a locked buf with the contents of the indicated block.
169 struct buf*
170 bread(uint dev, uint blockno)
171 {
172     struct buf *b;
173
174     b = bget(dev, blockno); //设备号和块号
175     //未命中，则先从磁盘中读取内存
176     if(!b->valid) {
177         virtio_disk_rw(b, 0);
178         b->valid = 1;
179     }
180     return b;
181 }

```

在 bwrite()中，先判断是否持有当前的睡眠锁，把当前缓冲区的内容 b 写入磁盘中。

```

183 //将缓冲内容写入磁盘，必须持有这块的睡眠锁
184 // Write b's contents to disk. Must be locked.
185 void
186 bwrite(struct buf *b)
187 {
188     if(!holdingsleep(&b->lock)) //是否持有睡眠锁
189         panic("bwrite");
190     virtio_disk_rw(b, 1);
191 }

```

brelse() bpin()和 bunpin()就是将锁换成指定桶的锁。brelse()释放前要检查当前线程是否持有缓冲区的睡眠锁，释放 buffer 的睡眠锁，并对整个 bcache 新增自旋锁，对引用计数进行减 1，删除缓冲区 b，并加入到 bcache 中。

```

193 // Release a locked buffer. //释放一个缓冲，必须持有该缓冲的睡眠锁
194 // Move to the head of the most-recently-used list.
195 void
196 brelse(struct buf *b)
197 {
198     if(!holdingsleep(&b->lock)) //是否持有缓冲区的睡眠锁
199         panic("brelse");
200     int bucket = hash(b->blockno);
201     releasesleep(&b->lock); //释放buffer的睡眠锁
202
203     acquire(&bcachebucket[bucket].lock); //对bcache加自旋锁
204     b->refcnt--;
205     //删除该缓冲区b，将b加入到LRU的bcache中
206     /*
207     if (b->refcnt == 0) {
208         // no one is waiting for it.
209         b->next->prev = b->prev; //先把b从链表中删除，更新缓冲区的指针
210         b->prev->next = b->next;
211         b->next = bcache.head.next; //将b插入到链表头部
212         b->prev = &bcache.head;
213         bcache.head.next->prev = b;
214         bcache.head.next = b;
215     }
216     */
217     release(&bcachebucket[bucket].lock);
218 }

```



```
219  /*
220  void
221  bpin(struct buf *b) {
222      acquire(&bcache.lock);
223      b->refcnt++;
224      release(&bcache.lock);
225  }
226  */
227
228  void
229  bpin(struct buf *b){
230      int bucket = hash(b->blockno); //哈希寻找桶
231      acquire(&bcachebucket[bucket].lock);
232      b->refcnt++;
233      release(&bcachebucket[bucket].lock);
234  }
235
236  /*
237  void
238  bunpin(struct buf *b) {
239      acquire(&bcache.lock);
240      b->refcnt--;
241      release(&bcache.lock);
242  }*/
243
244  void
245  bunpin(struct buf *b){
246      int bucket = hash(b->blockno); //哈希寻找桶
247      acquire(&bcachebucket[bucket].lock);
248      b->refcnt--;
249      release(&bcachebucket[bucket].lock);
250  }
```

进行 bcachetest 测试:

```
xv6 kernel is booting
```

```
hart 2 starting
```

```
hart 1 starting
```

```
init: starting sh
```

```
$ bcachetest
```

```
start test0
```

```
test0 results:
```

```
--- lock kmem/bcache stats
```

```
--- top 5 contended locks:
```

```
lock: virtio_disk: #test-and-set 14634514 #acquire() 1119
```

```
lock: proc: #test-and-set 3972325 #acquire() 192135
```

```
lock: proc: #test-and-set 3727329 #acquire() 192137
```

```
lock: proc: #test-and-set 3446821 #acquire() 192136
```

```
lock: proc: #test-and-set 3357748 #acquire() 189835
```

```
tot= 0
```

```
test0: OK
```

```
start test1
```

```
test1 OK
```

进行 usertests 测试:

```
$ usertests
usertests starting
test MAXVApus: usertrap(): unexpected scause 0x000000000000000f pid=13
    sepc=0x00000000000002204 stval=0x0000000400000000
usertrap(): unexpected scause 0x000000000000000f pid=14
    sepc=0x00000000000002204 stval=0x0000000800000000
usertrap(): unexpected scause 0x000000000000000f pid=15
    sepc=0x00000000000002204 stval=0x0000001000000000
```

```
test opentest: OK
test writetest: OK
test writebig: OK
test createtest: OK
test openiput: OK
test exitiput: OK
test iput: OK
test mem: OK
test pipel: OK
test killstatus: OK
test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
```

五、总结

为了解决锁竞争，要拆分单个资源为多个资源，并使用独立的锁进行保护。通过本实验学习了自旋锁和睡眠锁的实现机制，优化了 block cache 争用，改进 xv6 操作系统的缓冲区缓存 buffer cache，将原始的双向链表改为哈希桶，每一个桶内放置一些 buffer，并且每一个桶都有一个独立的锁。当使用 bget 获取 buffer 时，只需要将磁盘块的编号进行 hash，返回一个整数作为哈希表内桶的编号，并在桶中找到指定的 buffer。这样可以减少在多进程频繁使用文件系统时对 bcache.lock 的争用。